Nombre y apellido: _____

Ejercicios (puntaje)			Nota
1 (3 pts)	2 (3.5 pts)	3 (3.5 pts)	

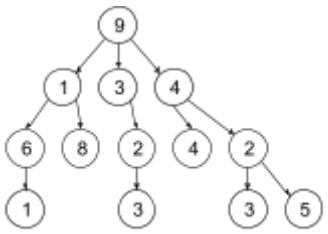
CONSIDERACIONES: RESOLVER CADA EJERCICIO EN UNA HOJA DIFERENTE. LOS EJERCICIOS QUE NO ESTÉN CORRECTOS EN UN 50% NO SUMARÁN PUNTOS PARA LA NOTA FINAL. PARA CADA EJERCICIO DEBE **DEFINIR EL TIPO DE DATO** DE CADA TDA UTILIZADO. **EN TODOS LOS CASOS QUE UTILICE ESTRUCTURAS** QUE NO SEAN TDAs "ESTÁNDAR" **DEBE DEFINIR LOS STRUCTS**.

Ejercicio 1: SIN USAR TDA, desarrollar cada una de las siguientes funciones necesarias para **implementar árboles AVL.**

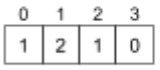
```
int height (btn* nodo); // Devuelve la altura de un nodo (el árbol nulo tiene altura -1).
int balanceFactor (btn* nodo); // Devuelve el factor de balanceo del nodo.
int leftRotation (btn** nodo); // Rota a izquierda un nodo de árbol binario de búsqueda.
```

Ejercicio 2: Desarrollar una función (y todas las necesarias) para que **dado un árbol n-ario devuelva un TDA *vector* con la cantidad de nodos por nivel cuyos hijos suman más de un valor N**.

Ejemplo:



Árbol de entrada y N= 5



TDA *Vector* de salida

Encabezado de la función principal: `vector* node_level_if (ntn* root, int N);`

NOTA: NO debe asumir que cada nodo contiene calculado el nivel como atributo, si se define como atributo **debe calcularlo**.

Ejercicio 3: SIN USAR TDA graph Dado un digrafo implementado con **listas de adyacencia**, crear las funciones necesarias para **generar un nuevo grafo transpuesto**.



Encabezado de la función principal: `<estructura grafo>* graph_transpose ( <estructura grafo>* g );`